PROYECTO INTERMODULAR DAM2 · ASIX2 · SIMIX2

AhorraTenergia

Memoria del proyecto

Sprint 1 y Sprint 2

Prototipo de gestión energética doméstica basado en la simulación de consumo, producción solar, batería y red eléctrica, integrando ESP32, ASP.NET Core y React.

Integrantes

Raúl Sánchez Ortiz · Hebert Alberto Calderon Torres · Noa Abril Casas

Periodo de los Sprints

Sprint 1: 6–20 de octubre de 2026 · Sprint 2: entrega 12 de noviembre de 2026

Repositorio

`https://github.com/rrrarrra9/AhorraTenergia`

Edición consolidada para el cierre del Sprint 2, actualizada el 11/11/2026. Incluye el arranque del proyecto, el cierre real del Sprint 1 y el primer incremento funcional integrado del MVP.

Índice general

Presentación IV

1. Introducción, problema y objetivos **1**

1.1. Introducción . . . 1
1.2. Problema o necesidad detectada . . . 1
1.3. Usuario objetivo . . . 1
1.4. Propuesta de valor . . . 1
1.5. Objetivo general . . . 1
1.6. Objetivos específicos . . . 2
1.7. Alcance del Sprint 1 . . . 2
1.8. Alcance del Sprint 2 . . . 2

2. Gestión del proyecto con GitHub **3**

2.1. Repositorio y estructura . . . 3
2.2. Flujo de trabajo . . . 3
2.3. Roles del equipo . . . 4
2.4. Definition of Done . . . 4
2.5. Pull Requests relevantes . . . 4

3. Sprint 1: arranque, planificación y cierre **5**

3.1. Objetivo del Sprint 1 . . . 5
3.2. Entregables principales . . . 5
3.3. Sprint Review del 20/10/2026 . . . 5
3.4. Retrospectiva del Sprint 1 . . . 6
3.5. Cierre documental del Sprint 1 . . . 6

4. Sprint 2: planificación y contrato técnico **7**

4.1. Objetivo inicial . . . 7
4.2. Contrato común de datos . . . 7
4.3. Convenciones . . . 7
4.4. Datos generados por ESP32 . . . 7
4.5. Datos gestionados por el Backend . . . 8
4.6. Ejemplo de telemetría ESP32 → Backend . . . 8

4.7. Interpretación de batería y red . . . 8
4.8. API mínima acordada . . . 9
4.9. Modelo de datos previsto para Supabase . . . 9

5. Implementación del Sprint 2 10
5.1. Backend ASP.NET Core . . . 10
5.1.1. Modelo de datos . . . 10
5.1.2. Almacenamiento temporal . . . 10
5.1.3. Validaciones de telemetría . . . 10
5.1.4. Estado actual e histórico . . . 11
5.1.5. Estado del servicio . . . 11
5.2. ESP32 y C++ . . . 11
5.2.1. Funciones implementadas . . . 11
5.2.2. Pruebas en entorno y placa física . . . 11
5.2.3. Seguridad de configuración . . . 12
5.3. Dashboard React . . . 12
5.3.1. Información mostrada . . . 12
5.3.2. Estados de interfaz . . . 12
5.3.3. Comunicación con la API . . . 12
5.3.4. Pendiente trasladado . . . 13

6. Integración, pruebas y resultados 14
6.1. Flujo integrado demostrado . . . 14
6.2. Pruebas del Backend . . . 14
6.3. Pruebas ESP32 → Backend . . . 14
6.4. Pruebas Backend → React . . . 14
6.5. Resultado frente al objetivo inicial . . . 15

7. Planificación, seguimiento y cierre del Sprint 2 16
7.1. Issues principales . . . 16
7.2. Trabajo trasladado al siguiente Sprint . . . 16
7.2.1. Issue #15 — Supabase PostgreSQL . . . 16
7.2.2. Issue #43 — Última actualización en React . . . 16
7.3. Desviaciones de proceso . . . 16
7.4. Cierre del Planning . . . 17

8. Backlog inicial, metodología y riesgos 18

8.1. Backlog documental inicial 18
8.2. Metodología de trabajo 19
8.3. Riesgos identificados 19

9. Seguridad y decisiones técnicas **20**
9.1. Principios de seguridad aplicados 20
9.2. Separación de responsabilidades 20
9.3. Limitaciones deliberadas del prototipo 20

10. Entregables y evidencias **21**
10.1. Matriz de entregables principales 21
10.2. Trazabilidad Sprint 2 21
10.3. Criterio de evidencia 21

Conclusión **22**

Presentación

AhorraTenergia es un proyecto intermodular orientado al desarrollo de un prototipo de gestión energética doméstica. El sistema representa mediante datos simulados la relación entre consumo eléctrico, producción solar, batería, red eléctrica y precio de la electricidad. El objetivo no es actuar físicamente sobre una instalación eléctrica real, sino construir una arquitectura de software y electrónica que permita visualizar, validar y analizar ese comportamiento de forma segura.

Durante el Sprint 1 se establecieron la organización del equipo, el repositorio, el backlog, el tablero de trabajo, la arquitectura inicial y la primera versión de la memoria. El Sprint 1 finalizó posteriormente con su Sprint Review y Retrospectiva documentadas.

Durante el Sprint 2 se construyó el primer incremento funcional del proyecto. Se definió un contrato de datos común, se implementó una API ASP.NET Core, se desarrolló la telemetría simulada con ESP32 y se implementó un dashboard React conectado al Backend.

Estado consolidado a 11/11/2026: el flujo demostrado durante el Sprint 2 es `ESP32` $\rightarrow$ `ASP.NET Core` $\rightarrow$ `React`. La persistencia mediante Supabase PostgreSQL, prevista inicialmente en el flujo completo, se ha trasladado al siguiente Sprint mediante el Issue #15. La visualización de `timestamp` como "Última actualización" en React se ha trasladado mediante el Issue #43.

CAPÍTULO 1

Introducción, problema y objetivos

1.1 Introducción

AhorraTenergia busca representar de forma comprensible el comportamiento energético de una vivienda que dispone de consumo eléctrico, producción fotovoltaica, batería y conexión a la red. El proyecto se desarrolla como prototipo educativo y utiliza datos simulados para evitar cualquier actuación sobre instalaciones eléctricas reales.

La solución combina una placa ESP32, un Backend ASP.NET Core, un dashboard React y una base de datos Supabase prevista para la persistencia. El Backend actúa como componente central de comunicación y validación.

1.2 Problema o necesidad detectada

En una vivienda con autoconsumo solar, batería y conexión a la red, el origen y destino de la energía cambian a lo largo del día. La producción fotovoltaica puede ser superior o inferior al consumo, la batería puede cargarse o descargarse y la vivienda puede importar o exportar energía a la red.

Además, el precio de la electricidad puede influir en futuras decisiones automáticas. Sin una vista conjunta de estas variables resulta difícil comprender el estado energético de la vivienda y estudiar reglas de gestión.

1.3 Usuario objetivo

El usuario principal es una persona que quiere comprender mejor el consumo energético de una vivienda, especialmente si dispone de paneles solares y batería. El proyecto también tiene un objetivo académico: demostrar la integración entre desarrollo web, Backend, bases de datos y electrónica programable.

1.4 Propuesta de valor

AhorraTenergia centraliza las variables energéticas principales y permite representar de forma visual el estado de una vivienda. La arquitectura está diseñada para evolucionar posteriormente hacia persistencia histórica, reglas automáticas, precios reales, autenticación y otros componentes del producto final.

1.5 Objetivo general

Diseñar, desarrollar y demostrar un prototipo capaz de recibir telemetría energética simulada, validarla, consultarla mediante una API y representarla en un dashboard, manteniendo una

arquitectura preparada para incorporar persistencia y reglas de gestión en iteraciones posteriores.

1.6 Objetivos específicos

1. Definir una arquitectura común para ESP32, ASP.NET Core, React y Supabase.
2. Establecer un contrato de datos único con nombres, unidades y formatos compartidos.
3. Generar telemetría energética simulada desde una ESP32.
4. Implementar una API REST para recibir, validar y consultar mediciones.
5. Representar el estado energético actual en un dashboard React.
6. Representar un histórico básico de consumo y producción solar.
7. Preparar el modelo de datos para su futura persistencia en Supabase PostgreSQL.
8. Mantener trazabilidad mediante Issues, ramas, commits, Pull Requests y documentación.
9. Evitar la exposición de credenciales, tokens, claves privadas o datos personales reales.

1.7 Alcance del Sprint 1

El Sprint 1 se centró en el arranque y planificación inicial del proyecto. Sus objetivos fueron establecer el repositorio, la organización del equipo, el backlog, el tablero Kanban, la arquitectura inicial, el diseño preliminar del dashboard y la primera versión de la memoria técnica.

1.8 Alcance del Sprint 2

El Sprint 2 tuvo como objetivo construir el primer incremento funcional integrado. El objetivo inicial contemplaba el flujo:

$$\texttt{ESP32} \rightarrow \texttt{ASP.NET Core} \rightarrow \texttt{Supabase} \rightarrow \texttt{ASP.NET Core} \rightarrow \texttt{React}$$

Durante el desarrollo se decidió aplazar la persistencia en Supabase al siguiente Sprint. Por tanto, el flujo efectivamente demostrado fue:

$$\texttt{ESP32} \rightarrow \texttt{ASP.NET Core} \rightarrow \texttt{React}$$

El Backend almacena temporalmente las mediciones en memoria. Esta decisión permite cerrar un incremento funcional verificable sin presentar como terminada una integración que todavía no se ha implementado.

CAPÍTULO 2

Gestión del proyecto con GitHub

2.1 Repositorio y estructura

El repositorio de referencia es:

https://github.com/rrrarrra9/AhorraTenergia

GitHub se utiliza para registrar tareas, gestionar ramas, versionar código, revisar Pull Requests y conservar las evidencias de cada Sprint.

```
AhorraTenergia/
  docs/
    memoria/
    actas/
    retrospectivas/
    adr/
    sprint2/
  src/
    backend/
    web/
    android/
    esp32/
  design/
  tests/
  README.md
```

La estructura contiene tanto componentes ya implementados como espacios reservados para trabajo posterior. La existencia de un directorio no implica por sí sola que su funcionalidad esté terminada.

2.2 Flujo de trabajo

El procedimiento acordado para las tareas relevantes es:

1. Crear o actualizar un Issue con objetivo, responsable y criterios de aceptación.
2. Crear una rama específica para la tarea.
3. Realizar commits descriptivos.
4. Abrir una Pull Request contra `main`.
5. Solicitar revisión a otro integrante.
6. Corregir las observaciones detectadas.
7. Obtener una aprobación formal `APPROVED` antes del merge cuando corresponda.
8. Fusionar los cambios y actualizar el Issue y la documentación.

2.3 Roles del equipo

Integrante	Responsabilidades principales durante el proyecto
Raúl	Backend ASP.NET Core, coordinación técnica y futura integración con Supabase.
Hebert	ESP32, integración, pruebas, documentación y trazabilidad.
Noa	Dashboard React, diseño de interfaz y apoyo a organización/documentación.

Los roles no impiden la revisión cruzada. Los tres integrantes participan en las decisiones de arquitectura y en la revisión de entregables.

2.4 Definition of Done

Una tarea se considera terminada cuando cumple su objetivo y sus criterios de aceptación, dispone de evidencias verificables, no contiene secretos, está documentada en un nivel adecuado y sigue el procedimiento de integración acordado. Si existe una desviación de proceso, debe registrarse expresamente.

2.5 Pull Requests relevantes

PR	Contenido	Estado / observación
#30	Plantillas de Sprint Review y Retrospectiva del Sprint 1	Fusionada con aprobación formal registrada.
#39	Contrato de datos del Sprint 2	Fusionada; documento utilizado como referencia común del Sprint.
#40	API base ASP.NET Core	Fusionada. La revisión quedó registrada como `COMMENTED`, no como `APPROVED`.
#41	Telemetría ESP32 e integración con Backend	Fusionada. La revisión quedó registrada como `COMMENTED`, no como `APPROVED`.
#42	Dashboard React Sprint 2	Fusionada después de una revisión con cambios solicitados y una aprobación formal final.

Mejora de proceso: durante el Sprint 2 se detectó que una revisión con comentario no equivale a una aprobación formal. La PR #42 aplicó correctamente el procedimiento: primero se solicitaron cambios, se corrigieron y después se registró `APPROVED` antes del merge.

CAPÍTULO 3

Sprint 1: arranque, planificación y cierre

3.1 Objetivo del Sprint 1

El Sprint 1 tuvo como finalidad establecer una base organizativa y técnica para el proyecto. Se trabajó principalmente sobre GitHub, documentación, arquitectura, backlog, diseño y metodología.

3.2 Entregables principales

Durante el Sprint 1 se prepararon, entre otros, los siguientes elementos:

- estructura inicial del repositorio;
- backlog y priorización;
- tablero Kanban;
- roles y Definition of Done;
- arquitectura inicial;
- diseño inicial del dashboard;
- memoria técnica en LaTeX;
- evidencias de commits y Pull Requests;
- documentación para Sprint Review y Retrospectiva.

3.3 Sprint Review del 20/10/2026

La Sprint Review se celebró el 20 de octubre de 2026 con la participación de Raúl, Noa y Hebert. Se mostraron el repositorio de GitHub, el GitHub Project, la memoria técnica, la arquitectura inicial y la propuesta gráfica del dashboard.

La principal observación recibida del profesorado se refería a la carpeta `tests/`. La comprobación posterior indicó que en ese momento contenía únicamente un archivo `.gitkeep`; por tanto, la existencia de la carpeta por sí sola no demostraba una vulnerabilidad. Posteriormente, en la Retrospectiva, el equipo acordó revisar la organización de las pruebas y evitar que en ellas aparezcan secretos o información sensible.

El acta se encuentra en:

docs/actas/2026-10-20-sprint-review.md

3.4 Retrospectiva del Sprint 1

La Retrospectiva también quedó documentada. El equipo identificó como aspectos positivos la organización del repositorio, la asignación de responsabilidades, el tablero Kanban, la memoria técnica, la arquitectura y el diseño inicial.

Las principales dificultades fueron el uso inicial de GitHub Projects, la falta de aprobación formal en algunas Pull Requests y la actualización tardía de parte de las evidencias.

Como acciones de mejora se acordó:

- revisar las Pull Requests antes del merge;
- mantener actualizado GitHub Projects;
- registrar las evidencias durante el Sprint y no únicamente al final;
- revisar la organización de las pruebas y evitar información sensible.

La Retrospectiva se encuentra en:

docs/retrospectivas/2026-10-20-retrospectiva-sprint1.md

3.5 Cierre documental del Sprint 1

El Issue #12, dedicado a las evidencias y documentación de cierre, terminó con sus 16 tareas completadas y quedó cerrado. De este modo, la Review y la Retrospectiva dejaron de ser pendientes y pasaron a formar parte de las evidencias reales del proyecto.

CAPÍTULO 4

Sprint 2: planificación y contrato técnico

4.1 Objetivo inicial

El Sprint 2 se planteó como el primer incremento funcional del MVP. El objetivo era demostrar que los principales componentes podían intercambiar, almacenar y representar información energética coherente.

El flujo inicialmente previsto fue:

$$\texttt{ESP32} \rightarrow \texttt{ASP.NET Core} \rightarrow \texttt{Supabase} \rightarrow \texttt{ASP.NET Core} \rightarrow \texttt{React}$$

La planificación quedó registrada en el Issue #34.

4.2 Contrato común de datos

Antes de integrar los componentes se definió un contrato común mediante el Issue #38 y el documento:

docs/sprint2/contrato-datos.md

El objetivo era evitar diferencias de nombres, unidades y estructuras entre ESP32, Backend y React.

4.3 Convenciones

Elemento	Convención
JSON	`camelCase`
C#	`PascalCase`
PostgreSQL / Supabase	`snake_case`
Potencias instantáneas	kW
Nivel de batería	%
Precio eléctrico	€/kWh
Fecha y hora	UTC, ISO 8601

4.4 Datos generados por ESP32

La ESP32 genera los siguientes campos:

- homeId;
- deviceId;
- timestamp;
- consumptionPowerKw;
- solarProductionPowerKw;
- batteryLevelPercent;
- batteryPowerKw;
- gridPowerKw.

Los identificadores utilizados durante las pruebas son:

```
homeId   = "home-01"
deviceId = "esp32-01"
```

4.5 Datos gestionados por el Backend

El Backend incorpora dos valores que no son enviados por la ESP32:

- electricityPriceEurKwh;
- systemMode.

En el incremento actual se utiliza un precio simulado de 0.18 €/kWh y el modo automatic.

4.6 Ejemplo de telemetría ESP32 → Backend

```
{
  "homeId": "home-01",
  "deviceId": "esp32-01",
  "timestamp": "2026-11-05T16:30:00Z",
  "consumptionPowerKw": 2.35,
  "solarProductionPowerKw": 1.80,
  "batteryLevelPercent": 68,
  "batteryPowerKw": 0.40,
  "gridPowerKw": 0.15
}
```

4.7 Interpretación de batería y red

Para batteryPowerKw:

- valor positivo: la batería entrega energía;
- valor negativo: la batería se carga;
- cero: sin intercambio.

Para `gridPowerKw`:

- valor positivo: importación desde la red;
- valor negativo: exportación hacia la red;
- cero: sin intercambio.

4.8 API mínima acordada

El contrato definió cuatro endpoints:

Endpoint	Responsabilidad
`POST /api/telemetry`	Recibir y validar una medición de la ESP32.
`GET /api/energy/current`	Devolver la última medición disponible.
`GET /api/energy/history ?limit=20`	Devolver un histórico limitado de mediciones.
`GET /api/health`	Comprobar que el Backend está operativo.

4.9 Modelo de datos previsto para Supabase

El modelo mínimo definido para la futura persistencia utiliza tres entidades:

- `homes`;
- `devices`;
- `energy_measurements`.

Las relaciones previstas son:

```
homes 1:N devices

devices 1:N energy_measurements
```

Este modelo fue definido durante el Sprint 2, pero su implementación real en Supabase se ha aplazado al siguiente Sprint.

CAPÍTULO 5

Implementación del Sprint 2

5.1 Backend ASP.NET Core

El Backend se encuentra en:

`src/backend/AhorraTenergia.Api/`

La API se desarrolló con ASP.NET Core y expone los cuatro endpoints acordados.

5.1.1 Modelo de datos

La clase `Datos` incluye las propiedades recibidas desde la ESP32 y las propiedades internas gestionadas por el Backend.

5.1.2 Almacenamiento temporal

Las mediciones se almacenan actualmente en una lista en memoria. Se incluyeron mediciones simuladas iniciales para permitir pruebas del estado actual y del histórico.

Este almacenamiento es deliberadamente temporal. No se presenta como sustituto definitivo de Supabase.

5.1.3 Validaciones de telemetría

El endpoint `POST /api/telemetry` comprueba, entre otros aspectos:

- que el cuerpo sea un objeto JSON;
- que existan los campos requeridos;
- que `homeId` y `deviceId` sean válidos;
- que el `timestamp` sea interpretable y esté expresado en UTC;
- que consumo y producción solar no sean negativos;
- que el nivel de batería esté entre 0 y 100;
- que las potencias sean valores numéricos finitos;
- que la ESP32 no intente enviar `electricityPriceEurKwh` ni `systemMode`.

Cuando una medición válida se registra, la API devuelve `201 Created`.

5.1.4 Estado actual e histórico

`GET /api/energy/current` localiza la medición más reciente por `timestamp`.

`GET /api/energy/history?limit=20` ordena las mediciones y devuelve las últimas entradas según el límite solicitado. Un límite inválido devuelve `400 Bad Request`.

5.1.5 Estado del servicio

`GET /api/health` devuelve una respuesta mínima con estado `ok`, permitiendo comprobar que el servicio está operativo.

5.2 ESP32 y C++

La parte de ESP32 se encuentra en:

`src/esp32/`

Se utilizó PlatformIO con Arduino Framework. La telemetría se genera de forma periódica y respeta los nombres y unidades definidos en el contrato común.

5.2.1 Funciones implementadas

La ESP32:

- genera consumo eléctrico simulado;
- genera producción solar simulada;
- genera nivel y potencia de batería;
- genera intercambio con la red;
- obtiene una referencia temporal mediante NTP y genera timestamps UTC;
- construye el JSON de telemetría;
- muestra información de prueba por puerto serie;
- envía peticiones HTTP a `POST /api/telemetry`;
- gestiona errores básicos de comunicación sin bloquear permanentemente el programa.

5.2.2 Pruebas en entorno y placa física

El desarrollo se comprobó tanto en entorno de simulación como con una ESP32 física. Durante la integración con el Backend se observaron respuestas HTTP `201`, lo que confirmó que la API aceptaba las mediciones enviadas.

Posteriormente, `GET /api/energy/current` permitió recuperar la medición más reciente recibida desde la ESP32.

5.2.3 Seguridad de configuración

La configuración local sensible se mantiene fuera del repositorio. El archivo local utilizado para secretos no debe versionarse y está contemplado por las reglas de exclusión del proyecto.

5.3 Dashboard React

El Frontend web se encuentra en:

`src/web/`

Está desarrollado con React y Vite.

5.3.1 Información mostrada

El dashboard representa:

- consumo eléctrico;
- producción solar;
- nivel de batería;
- potencia de batería;
- intercambio con la red;
- precio eléctrico simulado;
- modo del sistema;
- histórico de consumo y producción solar.

5.3.2 Estados de interfaz

Se contemplan cuatro estados:

- carga;
- datos disponibles;
- error;
- ausencia de datos.

5.3.3 Comunicación con la API

El servicio `energyService.js` permite trabajar con datos mock o con la API. Para el modo API consulta:

```
/api/energy/current
/api/energy/history?limit=20
```

Vite dispone de un proxy de desarrollo hacia el Backend local. La configuración puede documentarse mediante variables como:

```
VITE_ENERGY_SOURCE
VITE_API_BASE_URL
```

El repositorio incluye un archivo `.env.example` sin secretos.

5.3.4 Pendiente trasladado

La visualización de `timestamp` como "Última actualización" no quedó incluida en el cierre final de la PR #42. El equipo decidió no bloquear el cierre del Sprint por este ajuste y creó el Issue #43 para la siguiente iteración.

CAPÍTULO 6

Integración, pruebas y resultados

6.1 Flujo integrado demostrado

El resultado técnico demostrado durante el Sprint 2 es:

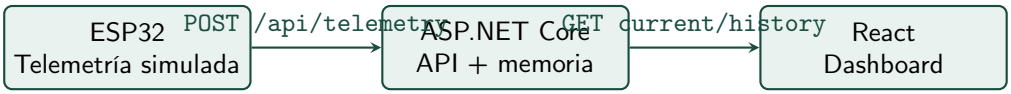


Figura 6.1: Flujo funcional comprobado al cierre del Sprint 2.

6.2 Pruebas del Backend

Se verificó el funcionamiento de:

- `POST /api/telemetry`;
- `GET /api/energy/current`;
- `GET /api/energy/history?limit=20`;
- `GET /api/health`.

También se comprobó que el Backend añade los valores internos de precio y modo y que rechaza mensajes que no cumplen las validaciones básicas.

6.3 Pruebas ESP32 → Backend

Durante las pruebas de integración la ESP32 envió telemetría al Backend y recibió respuestas `201 Created`. Las mediciones recibidas pudieron consultarse posteriormente mediante el endpoint de estado actual.

Esto permite demostrar que el contrato común fue suficiente para conectar la generación de telemetría con la API.

6.4 Pruebas Backend → React

El dashboard fue probado contra el Backend ASP.NET Core. Se comprobaron las consultas de estado actual e histórico y la representación visual de los datos.

Para el Frontend se ejecutaron además:

```
npm run lint
npm run build
```

La PR #42 registró estas comprobaciones como correctas antes del cierre.

6.5 Resultado frente al objetivo inicial

El objetivo inicial incluía Supabase. Sin embargo, la integración con PostgreSQL/Supabase no se implementó durante el Sprint 2.

Por tanto, no se afirma que se haya completado el flujo original completo. El resultado real es un incremento parcial pero funcional:

$$\texttt{ESP32} \rightarrow \texttt{ASP.NET Core} \rightarrow \texttt{React}$$

La persistencia se mantiene como trabajo pendiente y está registrada en el Issue #15.

CAPÍTULO 7

Planificación, seguimiento y cierre del Sprint 2

7.1 Issues principales

Issue	Actividad	Responsable	Estado
#38	Contrato común de datos y arquitectura	Equipo	Cerrado
#13	API base ASP.NET Core	Raúl	Cerrado
#14	Dashboard React	Noa	Cerrado
#18	Telemetría ESP32 e integración	Hebert	Cerrado
#34	Planning y cierre del Sprint 2	Equipo	Cerrado

Los Issues que finalmente formaron parte del incremento realizado durante el Sprint 2 quedaron cerrados con sus resultados documentados. El Issue #15, previsto inicialmente para Supabase, se trasladó al siguiente Sprint.

7.2 Trabajo trasladado al siguiente Sprint

7.2.1 Issue #15 — Supabase PostgreSQL

El Issue #15 permanece abierto, asignado a Raúl y sin milestone de Sprint 2. Su objetivo es implementar la persistencia energética real en Supabase PostgreSQL, incluyendo tablas, relaciones, scripts SQL, datos de prueba, consultas y acceso seguro desde el Backend.

7.2.2 Issue #43 — Última actualización en React

El Issue #43 permanece abierto, asignado a Noa. Su objetivo es recuperar `timestamp` desde el Backend y mostrarlo en el dashboard como "Última actualización".

7.3 Desviaciones de proceso

La PR #40 del Backend y la PR #41 de ESP32 fueron fusionadas sin una aprobación formal `APPROVED` previa. En ambos casos existieron revisiones registradas como `COMMENTED`.

El equipo decidió no revertir los merges, sino documentar la desviación y aplicar correctamente la regla en las siguientes Pull Requests.

La PR #42 mostró la mejora de proceso: se realizó una revisión, se solicitaron cambios, se corrigieron y finalmente se registró una aprobación formal antes del merge.

7.4 Cierre del Planning

El Issue #34 documenta que el resultado real del Sprint fue el flujo `ESP32` → `ASP.NET Core` → `React`, con Supabase aplazado. Este cierre evita presentar como realizada una parte que todavía no está implementada.

CAPÍTULO 8

Backlog inicial, metodología y riesgos

8.1 Backlog documental inicial

La planificación inicial identificó 25 tareas de referencia. Estos códigos son identificadores documentales y no deben confundirse automáticamente con los números de Issue de GitHub.

ID	Área	Resultado esperado	Prioridad
AT-01	Coordinación	Crear y mantener la estructura inicial del repositorio.	Alta
AT-02	Coordinación	Definir convenciones de ramas, commits e Issues.	Alta
AT-03	Documentación	Mantener la memoria del proyecto actualizada.	Alta
AT-04	Documentación	Registrar reuniones y acuerdos confirmados.	Alta
AT-05	Diseño	Preparar la propuesta inicial del dashboard.	Alta
AT-06	Diseño	Vincular el diseño al Issue y a evidencias.	Alta
AT-07	Arquitectura	Documentar componentes y relaciones.	Alta
AT-08	Backend	Crear la estructura de ASP.NET Core.	Alta
AT-09	Backend	Definir e implementar endpoints de la API.	Alta
AT-10	Backend	Definir modelos de variables energéticas.	Alta
AT-11	Backend	Preparar lógica de decisión simulada.	Alta
AT-12	Datos	Definir tablas de Supabase.	Alta
AT-13	Datos	Implementar persistencia de medidas.	Media
AT-14	Web	Crear la estructura base de React.	Alta
AT-15	Web	Diseñar componentes principales del dashboard.	Alta
AT-16	Web	Consumir datos de la API.	Media
AT-17	Android	Crear estructura inicial de la app Java.	Media
AT-18	Android	Diseñar vista inicial de consulta.	Media
AT-19	ESP32	Definir formato de datos simulados.	Alta
AT-20	ESP32	Integrar el intercambio con Backend.	Media
AT-21	Integración	Definir formatos de comunicación.	Media
AT-22	Pruebas	Definir escenarios energéticos simulados.	Alta
AT-23	Pruebas	Comprobar validación y coherencia de datos.	Media
AT-24	Seguridad	Evitar exposición de credenciales.	Alta
AT-25	Entrega	Revisar el estado del Sprint y preparar evidencias.	Alta

8.2 Metodología de trabajo

El equipo trabaja de forma incremental. Cada Sprint selecciona un conjunto limitado de tareas del Product Backlog, define un resultado demostrable y registra las evidencias en GitHub.

La secuencia habitual es:

$$\text{Issue} \rightarrow \text{rama} \rightarrow \text{desarrollo} \rightarrow \text{pruebas} \rightarrow \text{Pull Request} \rightarrow \text{revisión} \rightarrow \text{merge} \rightarrow \text{cierre}$$

8.3 Riesgos identificados

Riesgo	**Impacto**	**Mitigación**
Integración tecnológica	Incompatibilidades entre componentes	Utilizar un contrato común de datos.
Alcance excesivo	No terminar el incremento	Dividir el trabajo y aplazar elementos no esenciales.
Servicios externos	Dependencia de Supabase u otros servicios	Permitir almacenamiento temporal y datos de prueba.
Conflictos Git	Integraciones incorrectas	Ramas específicas y Pull Requests.
Revisión omitida	Incumplimiento de la DoD	Exigir `APPROVED` antes del merge.
Credenciales expuestas	Riesgo de seguridad	Archivos locales ignorados y ejemplos sin secretos.
Falta de evidencias	Dificultad para demostrar el trabajo	Registrar pruebas, Issues, PR y documentación durante el Sprint.

CAPÍTULO 9

Seguridad y decisiones técnicas

9.1 Principios de seguridad aplicados

Durante el desarrollo se mantiene la regla de no versionar:

- contraseñas;
- tokens;
- API keys;
- cadenas de conexión privadas;
- datos personales reales;
- archivos de configuración local que contengan secretos.

Cuando es necesario documentar variables de entorno se utilizan archivos de ejemplo sin credenciales, como `.env.example`.

9.2 Separación de responsabilidades

La arquitectura establece que React y ESP32 no deben acceder directamente a Supabase. El Backend ASP.NET Core actúa como mediador central.

Esto permite concentrar en la API las validaciones, el enriquecimiento de datos y, en futuras iteraciones, el acceso a persistencia y las reglas de negocio.

9.3 Limitaciones deliberadas del prototipo

El sistema actual:

- no controla relés ni elementos eléctricos reales;
- no conmuta físicamente entre red, solar y batería;
- no utiliza todavía precios eléctricos reales;
- no implementa autenticación;
- no implementa todavía la aplicación Android;
- no persiste todavía en Supabase;
- trabaja con una vivienda y un dispositivo de prueba.

Estas limitaciones forman parte de la estrategia de desarrollo por incrementos.

CAPÍTULO 10

Entregables y evidencias

10.1 Matriz de entregables principales

Entregable	Localización / referencia	Estado
Repositorio	GitHub AhorraTenergia	Operativo
Memoria LaTeX	`docs/memoria/`	Actualizada
Sprint Review 1	`docs/actas/`	Realizada
Retrospectiva 1	`docs/retrospectivas/`	Realizada
Contrato Sprint 2	`docs/sprint2/`	Versionado
Backend ASP.NET Core	`src/backend/`	Implementado
ESP32	`src/esp32/`	Implementado
Dashboard React	`src/web/`	Implementado
Persistencia Supabase	Issue #15	Pendiente
Timestamp en dashboard	Issue #43	Pendiente

10.2 Trazabilidad Sprint 2

Los principales Issues y Pull Requests del Sprint 2 permiten relacionar planificación, implementación y evidencias:

- Issue #38 y PR #39: contrato común de datos;
- Issue #13 y PR #40: Backend ASP.NET Core;
- Issue #18 y PR #41: ESP32 e integración HTTP;
- Issue #14 y PR #42: dashboard React;
- Issue #34: planificación y cierre del Sprint 2;
- Issue #15: persistencia Supabase trasladada;
- Issue #43: visualización de timestamp trasladada.

10.3 Criterio de evidencia

La memoria distingue entre lo planificado y lo realmente comprobado. Por esta razón, Supabase aparece como diseño previsto y trabajo pendiente, mientras que el flujo ESP32–Backend–React se presenta como integración demostrada.

Conclusión

El Sprint 1 permitió establecer la organización, documentación y arquitectura inicial de AhorraTenergia. Posteriormente, la Sprint Review y la Retrospectiva permitieron cerrar formalmente esa fase y detectar mejoras en el procedimiento de revisión y en el registro de evidencias.

El Sprint 2 transformó parte de esa planificación en un incremento funcional. El equipo definió un contrato común de datos, implementó una API ASP.NET Core, desarrolló la generación de telemetría con ESP32 y construyó un dashboard React conectado al Backend.

El resultado demostrado es el flujo:

$$\texttt{ESP32} \rightarrow \texttt{ASP.NET Core} \rightarrow \texttt{React}$$

La persistencia mediante Supabase no se completó durante el Sprint 2 y se mantiene abierta en el Issue #15. Del mismo modo, la visualización del timestamp en el dashboard se ha separado en el Issue #43. Esta decisión mantiene la documentación alineada con el estado real del proyecto y evita presentar como terminadas funcionalidades pendientes.

El siguiente incremento deberá centrarse principalmente en implementar la persistencia en Supabase, adaptar el Backend para almacenar y recuperar los datos desde PostgreSQL y completar los ajustes menores del dashboard. A partir de esta base podrán incorporarse progresivamente nuevas funcionalidades como autenticación, precios reales, reglas de gestión, múltiples viviendas y aplicación Android.

Fuentes de referencia

- Repositorio del proyecto: `https://github.com/rrrarrra9/AhorraTenergia`.
- Sprint Review 1: `docs/actas/`.
- Retrospectiva Sprint 1: `docs/retrospectivas/`.
- Contrato de datos Sprint 2: `docs/sprint2/`.
- Issues #13, #14, #15, #18, #34, #38 y #43.
- Pull Requests #39, #40, #41 y #42.